CS6886 Assignment 5
Optimising Attention on a CPU

Anirudhmoosad

2 November 2026

Introduction

The kernel is single-head attention,

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V,$$

with Q, K, V, O stored row-major as $N \times d$ FP32 matrices. Softmax is applied to each row. The driver times the median of five runs after one warm-up and reports

$$\mathrm{GFLOP/s} = \frac{4N^2 d}{t} \times 10^{-9},$$

which counts one multiply and one add in $QK^\top$ and the same again in the multiply by V. Every variant was checked with `make check`. A run passes when

$$\frac{\max |O - O_{\mathrm{naive}}|}{\max |O_{\mathrm{naive}}|} \leq 10^{-4}.$$

The compiler is Ubuntu clang 18.1.3 with `-O3 -march=native`. The OpenMP binary adds `-fopenmp`. No BLAS library is used.

1 Q1. Machine

The machine is a laptop with an Intel Core i5-11300H (Willow Cove, Tiger Lake-H), measured from Windows for the memory configuration and from the vendor specification for clocks and SIMD. The timings themselves were taken in Ubuntu on WSL2. WSL reports about 7.60 GB of RAM because that is the virtual-machine cap. The laptop has two 8.00 GB DDR4-3200 modules, so the memory size used below is 16.00 GB.

Table 1: Machine used for every measurement.

CPU	11th Gen Intel Core i5-11300H
Microarchitecture	Willow Cove (Tiger Lake-H)
Cores	4 physical, 8 logical; 4 performance, 0 efficiency
Clocks	3.10 GHz base, 4.40 GHz single-core, 3.10 GHz all-core
SIMD	AVX-512 (16 FP32 lanes) and AVX2 (8 FP32 lanes)
FMA	two 256-bit units; one 512-bit FMA uses both
Vector registers	32 zmm
L1 data	48.00 KiB per core
L2	1.25 MiB per core, private
L3	8.00 MiB, shared by all 4 cores
Cache line	64.00 B
RAM	16.00 GB DDR4-3200, 2 channels
Compiler	clang 18.1.3

On this core a 512-bit FMA produces 16 FP32 results and counts as two flops, and the two 256-bit units combine into one 512-bit FMA per cycle. The AVX2 form is the same peak: $8 \times 2 \times 2 = 32$ FP32 FLOP/cycle. So the peak is

$$\text{one core} = 4.40 \times 32 = 140.8 \text{ GFLOP/s},$$
$$\text{all cores} = 4 \times 3.10 \times 32 = 396.8 \text{ GFLOP/s}.$$

The all-core figure uses the 3.10 GHz base clock, which is the clock Windows reports under load. A short turbo can be higher.

DDR4-3200 transfers 8 bytes per channel per transfer, and both channels are populated:

$$3200 \times 8 \times 2/1000 = 51.2 \text{ GB/s}.$$

The ridge point is peak throughput divided by that bandwidth:

$$\text{one core} = 140.8/51.2 = 2.75 \text{ FLOP/byte},$$
$$\text{all cores} = 396.8/51.2 = 7.75 \text{ FLOP/byte}.$$

A kernel whose arithmetic intensity is below the ridge is limited by memory bandwidth. Above the ridge the ceiling is the flat compute roof.

2 Q2. Naive baseline

`make naive` produced the times in Table 2. Percentage of peak uses the one-core roof, 140.80 GFLOP/s.

Table 2: Naive attention. One thread.

N	d	time (ms)	GFLOP/s	% of 140.8
1024	64	133.938	2.00	1.4
2048	64	484.849	2.21	1.6
4096	64	2142.562	2.00	1.4
4096	128	8504.251	1.01	0.7

With d fixed, the flop count grows as N^2. From $N = 1024$ to $N = 2048$ the time grows by $484.849/133.938 = 3.62$, close to the factor of 4 in the flop count. From $N = 2048$ to $N = 4096$ at $d = 64$ the time grows by $2142.562/484.849 = 4.42$. The extra factor appears because the score matrix grows from 16.00 MB to 64.00 MB. At $N = 4096$ that matrix is eight times larger than the 8.00 MB L3 cache, so the reads and writes of S miss to DRAM.

Doubling d from 64 to 128 at $N = 4096$ only doubles the flop count, but the time grows by $8504.251/2142.562 = 3.97$ and the rate falls from 2.00 to 1.01 GFLOP/s. The $N \times N$ score matrix does not change size when d changes. The extra slowdown is the inner loop of $O = SV$, which walks V with a stride of d. A larger d makes that stride worse.

Almost all of the time is in the two matrix products. Softmax is only $O(N^2)$ exponentiations and divides, while each product is $O(N^2 d)$ flops. At 1–2% of the one-core peak the machine is not close to either roof. The naive loop is scalar, the SV loop is strided, and the score matrix is stored in full.

3 Q3. Optimisations

Each variant replaces only `attention()`. `main` is unchanged. Table 3 gives the single-thread times. Speedup is the naive time divided by the variant time.

Table 3: Single-thread variants. Speedup is against naive.

Variant	input	time (ms)	GFLOP/s	speedup	% of 140.8
loops	1024×64	46.466	5.78	2.9×	4.1
loops	2048×64	210.036	5.11	2.3×	3.6
loops	4096×64	893.625	4.81	2.4×	3.4
loops	4096×128	1709.823	5.02	5.0×	3.6
tiled	1024×64	25.350	10.59	5.3×	7.5
tiled	2048×64	106.363	10.10	4.6×	7.2
tiled	4096×64	464.163	9.25	4.6×	6.6
tiled	4096×128	816.050	10.53	10.4×	7.5
fused	1024×64	94.437	2.84	1.4×	2.0
fused	2048×64	402.052	2.67	1.2×	1.9
fused	4096×64	1851.806	2.32	1.2×	1.6
fused	4096×128	4146.230	2.07	2.1×	1.5
best	1024×64	16.801	15.98	8.0×	11.3
best	2048×64	72.806	14.75	6.7×	10.5
best	4096×64	413.171	10.40	5.2×	7.4
best	4096×128	565.603	15.19	15.0×	10.8

Figure 1 shows the same comparison, with the OpenMP bar taken at 8 threads.

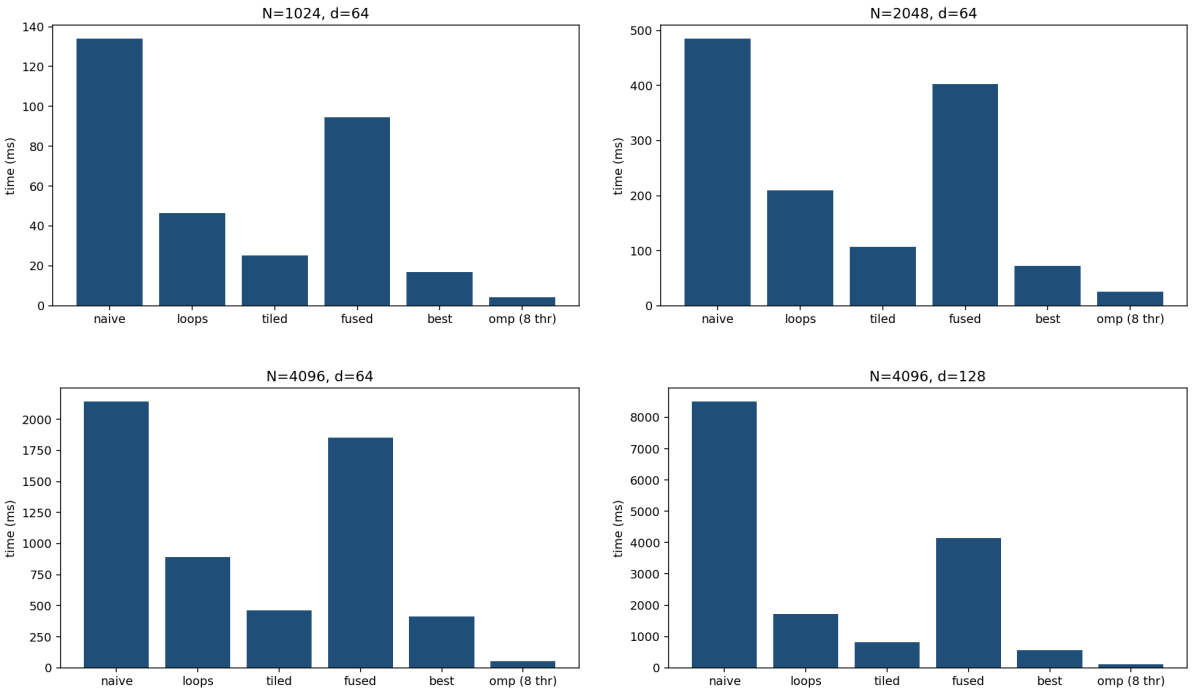


Figure 1: Median time of each variant. The single-thread kernels use one thread. The `omp` bar is the 8-thread run.

(a) Loop reorder

The $QK^\top$ loop was already a contiguous dot product. The $O = SV$ loop was not: its inner index stepped through column k of a row-major V, a stride of d floats. That loop was replaced by a zero of O followed by

$$O_{i,:} \mathrel{+}= S_{ij}\, V_{j,:},$$

so the inner index runs along a contiguous row of O and of V. `clang -Rpass=loop-vectorize` reports that this inner loop is vectorized with width 8 and interleave count 4. Width 8 is a 256-bit AVX2 register, eight FP32 values. Interleave 4 keeps four of those vectors in flight, 32 floats per step. On this CPU one 512-bit FMA uses both FMA units, so the 256-bit form has the same 32 FLOP/cycle peak.

The largest gain is at $d = 128$ (5.0×), where the old stride was the most harmful. The $N \times N$ matrix is still written and read, so at $N = 4096$ the kernel remains far below the one-core roof. `make check` reports error 0 on every input: for a fixed output element the sum over j is still taken in the same order.

(b) Cache and register tiling

Scores are computed in blocks of 32×32. A panel of K is $32 \times d \times 4$ bytes, which is 8.00 KiB at $d = 64$ and 16.00 KiB at $d = 128$. Both fit in the 48.00 KiB L1 data cache and are reused across the 32 query rows of the block. Inside the block each element of Q is reused across four running sums, a_0 through a_3. Those four accumulators are the register tile. The SV loop is tiled the same way and keeps the contiguous k order from part (a).

Softmax still needs a finished row, so it runs after every tile of S has been written. Tiling is about 1.8–2.1× faster than the reordered loops and 4.6–10.4× faster than naive. The errors

are about 5×10^{-7} to 7×10^{-7}, from adding the four-wide dot in a different order. The limit is unchanged: at $N = 4096$ the score matrix is 64.00 MB and does not fit in L3.

(c) Fused online softmax

This variant never allocates S. For each query row it keeps a running maximum m, a running sum ℓ, and a length-d accumulator. One key at a time,

$$\begin{aligned} m' &= \max(m, s), \\ \alpha &= e^{m-m'}, \qquad p = e^{s-m'}, \\ \ell &\leftarrow \alpha\ell + p, \\ \mathrm{acc} &\leftarrow \alpha\,\mathrm{acc} + p\,V_{j,:}, \end{aligned}$$

and $m \leftarrow m'$. After the row, $O_{i,:} = \mathrm{acc}/\ell$. This is the same softmax. The rescaling makes the running values match a row that had been normalised at the end.

The result passes, with errors about 10^{-6}, but it is only 1.2–2.1× faster than naive and about four times slower than tiled. Removing the 64.00 MB matrix is not enough. K and V are read again for every query. At $N = 4096$, $d = 64$ each of those matrices is 1.00 MB, far larger than the 48.00 KiB L1, so every row reloads them. `expf` also sits inside the inner loop, on the same dependency chain as the dot product, so clang cannot issue a clean vector FMA stream. The new limit is that repeated read of K and V, together with the exponential.

(d) Best single-thread kernel

The fastest single-thread kernel keeps the score matrix, because the fused inner loop lost to tiled. Blocks are 64×64. Four dots share one loaded element of Q, and the inner k loop is marked

```
#pragma clang loop vectorize(enable) interleave_count(4)
```

so the dot product is compiled as 256-bit FMAs rather than left as a scalar reduction. `restrict` on the four pointers tells clang that Q, K, V, O do not alias.

This is the fastest one-thread code: 5.2–15× over naive and 1.1–1.5× over tiled, at 7–11% of the one-core peak. The $d = 128$ input reaches a higher rate than $d = 64$ at the same N (15.19 against 10.40 GFLOP/s) because the flop count doubles while the score matrix stays 64.00 MB. The errors are about 10^{-6}. The remaining limit is that matrix, plus a reduction of length only 64 or 128, which does not fill the FMA pipeline the way a long GEMM does.

4 Q4. OpenMP

`attn_omp.c` is the best kernel with one change of schedule. The outer loop over query blocks of 64 rows is

```
#pragma omp parallel for schedule(static)
```

Each iteration computes the score rows for its block, the softmax of those rows, and the matching rows of O, before moving on. Rows of S and of O do not overlap, so there is no lock. Q, K, and V are only read. `static` gives each thread a contiguous set of blocks. A row is $d \times 4$ bytes, 256 or 512, which is several cache lines, so neighbouring threads do not share a cache line of O.

Doing softmax and SV inside the block, while those score rows are still hot, also helps at one thread. At $N = 4096$, $d = 64$ the one-thread OpenMP run takes 235.00 ms, against 413.00 ms for `best`.

Speedup is the one-thread time of this same binary divided by the time at p threads. Efficiency is that speedup divided by p. The machine has four physical cores, so the thread counts are 1, 2, 4, and 8.

Table 4: OpenMP at $N = 4096$, $d = 64$. Speedup is against the 1-thread run of `omp`.

threads	time (ms)	GFLOP/s	speedup	efficiency
1	235.470	18.24	1.00×	100%
2	120.925	35.52	1.95×	97%
4	93.297	46.04	2.52×	63%
8	55.446	77.46	4.25×	53%

Table 5: Speedup and efficiency on the other three inputs.

	$N = 1024$, $d = 64$		$N = 2048$, $d = 64$		$N = 4096$, $d = 128$	
threads	speedup	eff.	speedup	eff.	speedup	eff.
2	1.82×	91%	1.40×	70%	1.98×	99%
4	2.26×	56%	2.29×	57%	2.58×	64%
8	3.09×	39%	2.08×	26%	3.00×	37%

Two threads are almost twice as fast on the large inputs. Four threads reach only 2.3–2.6×. Part of that is the clock: one thread can boost to 4.40 GHz, while four cores together sit near the 3.10 GHz base, and the four cores share one memory bus and the 8.00 MB L3. Eight threads are still only four cores, with two hardware threads on each core. They share one pair of FMA units. One thread of this kernel is only about 15% of the one-core peak, so it spends much of its time waiting on the score matrix. The second thread on a core can run during that wait. That is why $N = 4096$, $d = 64$ climbs from 2.52× at 4 threads to 4.25× at 8 threads. On $N = 2048$ the 4-thread run is the faster one (22.80 ms against 25.20 ms at 8 threads): the extra threads only add contention.

The fastest point is 77.50 GFLOP/s, about 20% of the 396.80 GFLOP/s all-core peak. The score matrix is still being written.

5 Q5. Roofline

Arithmetic intensity uses the same $4N^2d$ flops as the timer. At $N = 4096$, $d = 64$ that is 4.295×10^9 flops. The denominator is DRAM traffic, with two cache rules. Q, K, V, and O are 1.00 MB each and fit in the 8.00 MB L3, so each is counted once. The score matrix is $4096^2 \times 4 = 64.00$ MB and does not fit in L3, so every pass that misses is counted.

For naive, loops, tiled, and best the code writes all of S, then softmax reads it and writes it back, then SV reads it again. A softmax row is 16.00 KB and stays in L1 while the three short passes over that row run, so those passes are one DRAM read and one dirty writeback. Together

with Q, K, V, O,

$$\text{bytes} = 16Nd + 16N^2 = 273.00\,\text{MB}, \qquad \text{AI} = \frac{4N^2 d}{16Nd + 16N^2} = 15.75\ \text{FLOP/byte}.$$

The one-thread OpenMP kernel applies softmax and SV to a 64-row block before the next block. That block is $64 \times 4096 \times 4 = 1.00\,\text{MB}$, which fits in the 1.25 MiB L2, so S is written back once and the later reads hit L2:

$$\text{bytes} = 16Nd + 4N^2 = 71.00\,\text{MB}, \qquad \text{AI} = 60.24\ \text{FLOP/byte}.$$

At 8 threads, two threads share each L2. Two 1.00 MB blocks do not fit in 1.25 MiB, so S is written and then read from DRAM:

$$\text{bytes} = 16Nd + 8N^2 = 138.00\,\text{MB}, \qquad \text{AI} = 31.03\ \text{FLOP/byte}.$$

Fused attention stores no score matrix. K and V together are 2.00 MB and stay in L3, because nothing else large is being written. DRAM traffic is one read of Q, K, V and one write of O:

$$\text{bytes} = 16Nd = 4.00\,\text{MB}, \qquad \text{AI} = N/4 = 1024\ \text{FLOP/byte}.$$

Table 6: Roofline inputs at $N = 4096$, $d = 64$. Implied bandwidth is achieved GFLOP/s divided by arithmetic intensity.

Variant	AI (FLOP/byte)	GFLOP/s	implied GB/s	side of ridge
naive	15.75	2.00	0.13	compute
loops	15.75	4.81	0.31	compute
tiled	15.75	9.25	0.59	compute
best	15.75	10.40	0.66	compute
omp, 1 thread	60.24	18.24	0.30	compute
omp, 8 threads	31.03	77.46	2.50	compute
fused	1024	2.32	0.002	compute

The one-core ridge is 2.75 FLOP/byte and the all-core ridge is 7.75. Every point in Table 6 is to the right of its ridge, so the roofline ceiling is the flat compute roof: 140.80 GFLOP/s for one thread and 396.80 GFLOP/s for eight threads. None of the runs is near the 51.20 GB/s slope. The busiest run, 8-thread OpenMP, uses about 2.50 GB/s.

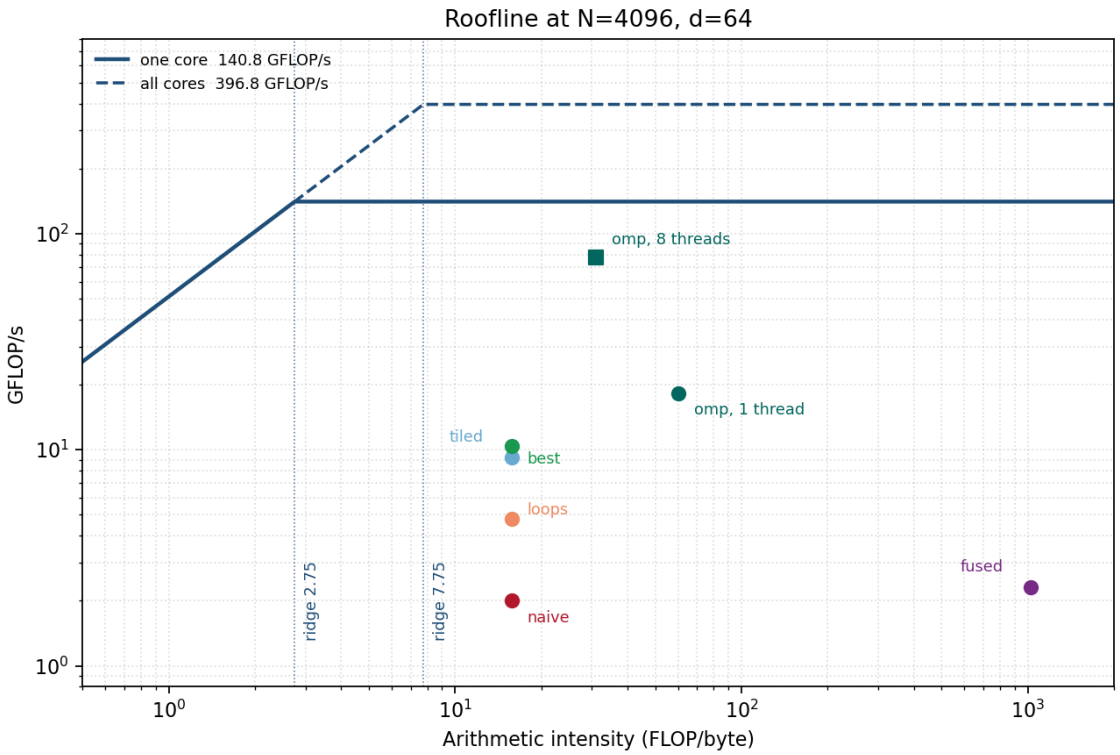


Figure 2: Roofline of the i5-11300H with every variant at $N = 4096$, $d = 64$. The solid roof is one core. The dashed roof is all four cores.

What limits each variant, given that placement:

- Naive, loops, tiled, and best share $x = 15.75$ and stack vertically. Naive at 2.00 GFLOP/s is scalar and strides through V. Loops at 4.80 GFLOP/s vectorizes the saxpy and still stores S. Tiled and best reach 9.3 and 10.40 GFLOP/s by reusing a K panel from L1 and vectorizing the dot product. Best is still about 7% of the one-core roof because d is short and every score is stored.
- Fused is the rightmost point and one of the lowest, at 2.30 GFLOP/s. Dropping S raises the intensity to 1024, and `expf` on every key keeps the FMA units idle. A higher intensity did not produce a higher rate.
- One-thread OpenMP is higher and further right than best, at 18.20 GFLOP/s, because the score block is reused from L2 instead of being written out and read back from DRAM.
- Eight-thread OpenMP is the highest point, at 77.50 GFLOP/s, about 20% of the all-core roof. The four cores share the L3 cache, and the extra four threads share the FMA units. The gap to 396.80 GFLOP/s is that incomplete use of the floating-point units, not the memory slope.